

[Day 1 종합 실습] 데이터 수집 미니 파이프라인

제출자 광주 1반 최재은 · 실습명 day1_pipeline · 작성일 2026-07-20

1. 프로젝트 개요

3개의 외부 API를 asyncio + httpx로 동시에 수집하고, Pydantic v2 모델로 타입과 값의 범위를 검증한다. 검증된 결과는 CSV와 Parquet로 저장하고 각 형식의 읽기·쓰기 시간을 비교한다.

구성	역할
pipeline.py	API 수집 → Pydantic 검증 → 저장·성능 비교 전체 실행
tests/test_pipeline.py	정상 날씨 데이터와 잘못된 강수확률 검증 테스트
requirements.txt	httpx, pandas, pyarrow, pydantic, pytest, ruff 관리
output/	collected_data.csv, collected_data.parquet 결과 저장

사용 API Open-Meteo(서울 3일 시간대별 기온·강수확률) / Countries.dev(대한민국 국가 정보) / ip-api(8.8.8.8 기반 지역 정보)

2. 실행 결과

(A) 파이프라인 실행 `python pipeline.py`

```
data-project/day1_pipeline on [main !] via v3.11.15 (.venv)
> python pipeline.py
[요청 시작] weather
[요청 시작] country
[요청 시작] ip
[응답 완료] ip: HTTP 200
[응답 완료] country: HTTP 200
[응답 완료] weather: HTTP 200

Pydantic 검증 성공: weather, country, ip

===== 저장 및 성능 비교 =====
저장된 데이터: 3행
CSV 쓰기: 0.008659초
Parquet 쓰기: 0.034558초
CSV 읽기: 0.001040초
Parquet 읽기: 0.032514초
CSV 파일: /Users/jaem/workspace/data-project/day1_pipeline/output/collected_data.csv
Parquet 파일: /Users/jaem/workspace/data-project/day1_pipeline/output/collected_data.parquet

===== 수집 결과 =====
weather: 정상 수집, 최상위 키 ['latitude', 'longitude', 'generationtime_ms', 'utc_offset_seconds', 'timezone']
country: 정상 수집, 최상위 키 ['area', 'cioc', 'flag', 'gini', 'name']
ip: 정상 수집, 최상위 키 ['status', 'country', 'countryCode', 'city', 'lat']

총 수집 API: 3개
전체 수집 시간: 1.2104초
3개 API 동시 수집 Checkpoint 통과
```

3개 API가 모두 HTTP 200으로 응답했으며 Pydantic 검증, CSV-Parquet 저장, 재로딩 건수 확인과 동시 수집 Checkpoint를 통과했다.

2. 실행 결과 (계속)

(B) 단위 테스트 `python -m pytest -v`

```
data-project/day1_pipeline on main [x!?] via 🐍 v3.11.15 (.venv)
> python -m pytest -v

===== test session starts =====
platform darwin -- Python 3.11.15, pytest-9.1.1, pluggy-1.6.0 -- /Users/jaem/workspace/data-project/.venv/bin/python
cachedir: .pytest_cache
rootdir: /Users/jaem/workspace/data-project/day1_pipeline
plugins: anyio-4.14.2
collected 2 items

tests/test_pipeline.py::test_weather_record_accepts_valid_data PASSED [ 50%]
tests/test_pipeline.py::test_weather_record_rejects_invalid_probability PASSED [100%]

===== 2 passed in 0.36s =====
```

정상 날씨 데이터가 통과하는지, 강수확률이 100을 초과할 때 `ValidationError`가 발생하는지를 검사했으며 2개 테스트가 모두 통과했다.

(C) 코드 스타일 검사 `ruff check` .

```
data-project/day1_pipeline on main [x!?] via 🐍 v3.11.15 (.venv)
> ruff check .
All checks passed!
```

Ruff 검사 결과 오류 없이 All checks passed를 확인했다.

3. CSV와 Parquet 성능 비교

항목	실행 결과
CSV 쓰기	0.008659초
Parquet 쓰기	0.034558초
CSV 읽기	0.001040초
Parquet 읽기	0.032514초

결과 해석 이번 실행에서는 데이터가 3행으로 매우 작기 때문에 CSV가 더 빠르게 측정되었다. Parquet는 스키마와 메타데이터 처리 비용이 있어 작은 데이터에서는 초기 비용이 상대적으로 크게 보일 수 있다. 따라서 한 번의 측정만으로 형식의 우열을 단정하지 않고 데이터 크기와 반복 측정을 함께 고려해야 한다.

4. Git 커밋 결과

```
data-project on  main via  v3.9.6
> git log --oneline -6
35e5c87 (HEAD -> main, origin/main) 종합실습 완료
581a6b6 종합실습 1차 완료
6eff53a 예외처리, pydantic 검증
59d9be0 .gitignore add
2e34f9c 파이썬 실습 시작
```

실습 시작, .gitignore 추가, 예외 처리-Pydantic 검증, 1차 완료, 최종 완료의 과정이 main 브랜치의 커밋 이력으로 기록되어 있다.

5. 코드 분석 및 본인 의견

5.1 구현에서 확인한 핵심

- ① 비동기 수집 각 API 호출을 작업으로 만들고 `asyncio.gather()`에서 함께 기다리므로, 순서대로 요청하는 방식보다 네트워크 대기 시간을 효율적으로 사용할 수 있다.
- ② Pydantic 검증 API 원본 필드를 `WeatherRecord`, `CountryRecord`, `IPRecord`에 명시적으로 연결했다. 위·경도, 강수량, 인구·면적, IP 주소처럼 의미가 분명한 값은 타입과 범위까지 검사한다.
- ③ 3일치 날씨 보존 `hourly` 배열의 첫 값만 선택하지 않고 시간, 기온, 강수량 목록 전체를 모델에 전달하여 서울 3일 시간대별 자료를 검증한다.
- ④ 저장 결과 확인 CSV와 Parquet를 쓴 뒤 다시 읽고 원본 DataFrame과 행 수가 같은지 `assert`로 확인해 저장 과정의 누락을 점검한다.

5.2 현재 코드의 장점과 개선 가능성

관점	정리
가독성	한 파일 안에서 수집→검증→저장 흐름을 순서대로 확인할 수 있어 학습용으로 이해하기 쉽다.
명확성	API마다 이름이 다른 필드를 Pydantic 모델 필드에 직접 연결하여 변환 관계가 분명하다.
예외 처리	검증·응답 구조 오류는 처리하지만 네트워크 장애에 대한 재시도 기능은 추후 추가할 수 있다.
테스트	필수 스키마 테스트는 통과했으며 향후 국가·IP 모델과 API 수집 모킹 테스트로 확장할 수 있다.
성능 측정	현재는 1회 측정이므로 반복 평균과 여러 데이터 크기를 사용하면 비교의 신뢰도를 높일 수 있다.